Setting Up R, Vimtex, and UltiSnips in Vim

Configuring a productive R and LaTeX editing environment in Vim on macOS

Ronald 'Ryy' G. Thomas

2025-06-15

Table of contents

1 Introduction **2**
- 1.1 Motivations . . . 2
- 1.2 Objectives . . . 3

2 Prerequisites and Setup **3**

3 What is a Vim-Based R and LaTeX Workflow? **4**

4 Getting Started **4**

5 The Complete .vimrc Configuration **6**

6 Filetype Plugin for R Markdown **10**

7 Filetype Plugin for R Scripts **11**

8 UltiSnips Configuration **12**

9 Practical Application **13**

10 R Markdown Template Structure **14**
- 10.1 Things to Watch Out For . . . 14
- 10.2 Lessons Learnt . . . 15
  - 10.2.1 Conceptual Understanding . . . 15
  - 10.2.2 Technical Skills . . . 16
  - 10.2.3 Gotchas and Pitfalls . . . 16
- 10.3 Limitations . . . 16
- 10.4 Opportunities for Improvement . . . 17

11 Wrapping Up **17**

12 See Also **17**

13 Reproducibility **18**

Figure 1: A productive terminal workspace with Vim, R, and LaTeX running side by side.

Setting the stage for a unified editing environment.

1 Introduction

I did not really know how to connect Vim, R, and LaTeX into a single productive workflow until I spent a weekend configuring vimtex and UltiSnips from scratch. Before that weekend, my workflow was fragmented: I wrote R code in one editor, switched to another for LaTeX, and lost time every time I moved between tools.

The frustration peaked when I realised I was spending more time switching contexts than actually writing code or prose. Every tool had its own shortcuts, its own conventions, and its own mental model. I needed a single environment that could handle R scripts, R Markdown documents, and LaTeX files without forcing me to leave Vim.

This post documents the complete configuration I arrived at after several rounds of trial and error. It covers the .vimrc plugin setup, filetype-specific configurations for R and R Markdown, UltiSnips snippet management, and a practical workflow example using the Palmer Penguins dataset.

1.1 Motivations

- I was tired of context-switching between RStudio, a LaTeX editor, and the terminal every time I worked on a statistical report.
- I wanted a single editor that could send R code to a live REPL, compile LaTeX documents, and expand code snippets without leaving the keyboard.
- My existing Vim setup had no awareness of R or LaTeX filetypes, so I had to configure everything from scratch.

- I needed ALE linting and auto-fixing for R code style consistency across projects.
- I wanted UltiSnips templates for R Markdown YAML headers so I could scaffold new analysis files in seconds.
- I had never configured ftplugin files before and wanted to understand how Vim dispatches settings by filetype.

1.2 Objectives

1. Install and configure vimtex, UltiSnips, ALE, and supporting plugins in a single .vimrc file.
2. Create filetype-specific configurations for R and R Markdown that open an R terminal and define useful keybindings.
3. Set up UltiSnips snippet expansion and popup menu navigation without conflicts between Tab mappings.
4. Walk through a practical example of editing an R Markdown file with the Palmer Penguins dataset inside this configured Vim environment.

I am documenting my learning process here. If you spot errors or have better approaches, please let me know.

The kind of focused environment where configuration work happens best.

2 Prerequisites and Setup

Before proceeding, you will need the following software installed on your macOS system:

- **Vim 8.1+** or **Neovim 0.5+** with terminal support
- **R 4.4+** from CRAN
- **MacTeX** from TUG
- **vim-plug** plugin manager, installed with:

```
curl -fLo ~/.vim/autoload/plug.vim \
  --create-dirs \
  https://raw.githubusercontent.com/\
junegunn/vim-plug/master/plug.vim
```

You should also have a working terminal emulator (iTerm2 or Kitty) and basic familiarity with Vim normal and insert modes. See the companion post "Setting up a minimal neovim…" for details on installing plugins with Neovim.

3 What is a Vim-Based R and LaTeX Workflow?

A Vim-based R and LaTeX workflow is a configuration that lets you write R code, R Markdown documents, and LaTeX files inside a single Vim session. Instead of switching between RStudio for data analysis and a dedicated LaTeX editor for typesetting, you use Vim as the unified interface.

Think of it as turning Vim into a lightweight IDE. The .vimrc file loads plugins for LaTeX compilation (vimtex), code snippet expansion (UltiSnips), and syntax linting (ALE). Filetype plugins detect whether you are editing an R script or an R Markdown file and automatically open an R terminal in a split pane. Keybindings let you send code to the REPL, inspect data objects, and render documents without leaving the editor.

The practical benefit is speed. Once configured, you can go from opening a file to running a complete analysis and compiling a PDF report without ever touching the mouse or switching applications.

4 Getting Started

The first step is to create or edit your `~/.vimrc` file with the full plugin configuration. The configuration below is a complete, working .vimrc that I use daily. It includes plugin declarations, global settings, and keybindings.

After saving the .vimrc, open Vim and run `:PlugInstall` to download and install all declared plugins.

Figure 2: A close-up of a keyboard and terminal screen showing configuration files being edited.

The iterative process of refining a configuration file one setting at a time.

5 The Complete .vimrc Configuration

The following is the full `~/.vimrc` file. It is organised into sections: plugin declarations with vim-plug, plugin-specific settings inline with each `Plug` call, and global editor settings at the end.

```
set runtimepath^=~/vimplugins
syntax enable
filetype plugin indent on
let mapleader = ","
let maplocalleader = " "

call plug#begin('~/vimplugins')

" -- Colour schemes --
Plug 'rakr/vim-one'
Plug 'mhartington/oceanic-next'
Plug 'rafi/awesome-vim-colorschemes'
Plug 'tpope/vim-vividchalk'

" -- Clipboard --
Plug 'jasonccox/vim-wayland-clipboard'

" -- Alignment --
Plug 'junegunn/vim-easy-align'

" -- Undo tree --
Plug 'mbbill/undotree'

" -- Terminal REPL integration --
Plug 'jpalardy/vim-slime'

" -- LaTeX --
Plug 'lervag/vimtex'
let g:vimtex_complete_close_braces=1
let g:vimtex_quickfix_mode=0

" -- Snippets --
Plug 'sirver/ultisnips'
let g:UltiSnipsExpandTrigger="<C-tab>"
let g:UltiSnipsJumpForwardTrigger="<tab>"
let g:UltiSnipsJumpBackwardTrigger="<S-tab>"
nnoremap <leader>U \
  <Cmd>call UltiSnips#RefreshSnippets()<CR>
nnoremap <leader>u :UltiSnipsEdit<cr>
```

```
" -- Linting and fixing --
Plug 'dense-analysis/ale'
let g:ale_virtualtext_cursor = 'disabled'
let g:ale_set_balloons = 1
highlight clear ALEErrorSign
highlight clear ALEWarningSign
let g:ale_sign_error = '●'
let g:ale_sign_warning = '.'
let g:ale_linters = {
\   'python': ['pylsp']
\ }
let g:ale_fix_on_save = 1
let g:ale_fixers = {
\   '*': ['remove_trailing_lines',
\         'trim_whitespace'],
\   'r': ['styler'],
\   'rmd': ['styler'],
\   'quarto': ['styler'],
\   'python': ['black','isort'],
\   'javascript': ['eslint'],
\ }
nnoremap <leader>n :ALENext<CR>

" -- Fuzzy finder --
Plug 'junegunn/fzf',
  \ { 'do': { -> fzf#install() } }
Plug 'junegunn/fzf.vim'
nnoremap <leader>z :Files<CR>
nnoremap <Leader>' :Marks<CR>
nnoremap <Leader>/ :BLines<CR>
nnoremap <Leader>b :Buffers<CR>
nnoremap <Leader>r :Rg<CR>
nnoremap <Leader>s :Snippets<CR>
tnoremap <leader>b <C-w>:Buffers<cr>
tnoremap <leader>r <C-w>:Rg<cr>
tnoremap <leader>z <C-w>:Files<cr>

" -- Navigation and editing --
Plug 'junegunn/vim-peekaboo'
Plug 'tpope/vim-unimpaired'
Plug 'tpope/vim-obsession'
Plug 'tpope/vim-repeat'
Plug 'tpope/vim-commentary'
autocmd FileType quarto
  \ setlocal commentstring=#\ %s
Plug 'tpope/vim-surround'
```

```
Plug 'justinmk/vim-sneak'
let g:sneak#label = 1
let g:sneak#s_next = 1
let g:sneak#use_ic_scs = 1
Plug 'machakann/vim-highlightedyank'

" -- Status line --
Plug 'vim-airline/vim-airline'
let g:airline#extensions#ale#enabled = 1
let g:airline#extensions#tabline#enabled = 1
let g:airline#extensions#fzf#enabled = 1

" -- R integration --
Plug 'rgt47/rgt-R'

" -- Completion --
Plug 'girishji/vimcomplete'

call plug#end()

" -- Editor settings --
if $COLORTERM == 'truecolor'
  set termguicolors
endif
set scrolloff=7
set iskeyword-=.
set completeopt=menu,menuone,popup,
  \noinsert,noselect
set complete+=k
set dictionary=/usr/share/dict/words
highlight Pmenu guifg=Black guibg=cyan
  \ gui=bold
highlight PmenuSel gui=bold guifg=White
  \ guibg=blue
set gfn=Monaco:h14
set encoding=utf-8
set lazyredraw
set autochdir
set number relativenumber
set clipboard=unnamed
set textwidth=80
set colorcolumn=80
set cursorline
set hlsearch
set incsearch
set ignorecase
set smartcase
```

```
set showmatch
set noswapfile
set hidden
set gdefault
set splitright
set wildmenu
set wildignorecase
set wildmode=list:full

" -- Popup menu navigation with Tab --
inoremap <expr> <tab>
  \ pumvisible() ? "\<C-n>" : "\<tab>"
inoremap <expr> <S-tab>
  \ pumvisible() ? "\<C-p>" : "\<S-tab>"

" -- Buffer and window navigation --
nnoremap <leader>o <C-w>:b1<CR>
nnoremap <leader>t <C-w>:b2<CR>
nnoremap <leader>h <C-w>:b3<CR>
nnoremap <leader><leader> <C-w>w
nnoremap <leader>a ggVG
nnoremap <leader>m vipgq
nnoremap <leader>f :tab split<cr>
nnoremap <leader>v :edit ~/.vimrc<cr>
nnoremap <localleader><leader> <C-u>
nnoremap <localleader><localleader> <C-d>
noremap - $
noremap : ;
noremap ; :
inoremap <F10> <C-x><C-k>
inoremap <F12> <C-x><C-o>
inoremap <silent> <Esc> <Esc>`^

" -- Terminal mode navigation --
tnoremap <F1> <C-\><C-n>
tnoremap <leader>o <C-w>:b1<CR>
tnoremap <leader>t <C-w>:b2<CR>
tnoremap <leader>h <C-w>:b3<CR>
tnoremap <leader><leader> <C-w>w
tnoremap lf ls()<CR>

" -- Auto-save and formatting --
au FocusGained * :let @z=@*
set updatetime=1000
autocmd CursorHold,CursorHoldI * update
autocmd FileType rmd
```

```
  \ setlocal commentstring=#\ %s
let $FZF_DEFAULT_OPTS =
  \ '--bind "ctrl-j:down,ctrl-k:up,'
  \ . 'j:preview-down,k:preview-up"'
set formatoptions-=c
  \ formatoptions-=r formatoptions-=o

" -- EasyAlign --
xmap ga <Plug>(EasyAlign)
nmap ga <Plug>(EasyAlign)

colorscheme one
set background=dark
```

After saving this file, open Vim and run:

```
:PlugInstall
```

Vim-plug will download and install every plugin declared between `plug#begin()` and `plug#end()`.

6 Filetype Plugin for R Markdown

Create the file `~/.vim/ftplugin/rmd.vim`. This file is loaded automatically whenever Vim opens a file with the `.Rmd` or `.rmd` extension. It defines terminal-mode shortcuts for rendering and sourcing, and normal-mode shortcuts for inspecting R objects under the cursor.

```
" ~/.vim/ftplugin/rmd.vim

" -- Terminal shortcuts for R Markdown --
tnoremap ZD quarto::quarto_render(
  \output_format = "pdf")<CR>
tnoremap ZO source("<C-W>"%")
tnoremap ZQ q('no')<C-\><C-n>:q!<CR>
tnoremap ZR render("<C-W>"%")<CR>
tnoremap ZT :!R -e 'render("<C-r>%",
  \ output_format="pdf_document")'<CR>
tnoremap ZS style_dir()<CR>
tnoremap ZX exit<CR>
tnoremap ZZ q('no')<C-\><C-n>:q!<CR>

" -- Object inspection under cursor --
nnoremap <localleader>d
  \ :let @c=expand("<cword>") \|
  \ :let @d="dim(".@c.")"."\n" \|
  \ :call term_sendkeys(
```

```
  \   term_list()[0], @d)<CR>
nnoremap <localleader>h
  \ :let @c=expand("<cword>") \|
  \ :let @d="head(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>
nnoremap <localleader>s
  \ :let @c=expand("<cword>") \|
  \ :let @d="str(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>
nnoremap <localleader>p
  \ :let @c=expand("<cword>") \|
  \ :let @d="print(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>
nnoremap <localleader>n
  \ :let @c=expand("<cword>") \|
  \ :let @d="names(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>

" -- Auto-open R terminal --
autocmd BufEnter *
  \ if &ft ==# 'rmd' && !exists('b:entered')
  \ | execute(
  \   'let b:entered = 1 | :ter ++rows=5 R')
  \ | endif
```

The object inspection mappings deserve explanation. When your cursor is on a variable name, pressing `<localleader>d` extracts the word under the cursor, wraps it in `dim()`, and sends the command to the R terminal. This lets you check dimensions, print heads, inspect structure, and view names of any R object without leaving normal mode.

7 Filetype Plugin for R Scripts

Create the file `~/.vim/ftplugin/r.vim`. This file is nearly identical to the R Markdown ftplugin but uses the alternate file register (`#` instead of `%`) for sourcing commands, which is appropriate when the R script is not the current buffer.

```
" ~/.vim/ftplugin/r.vim

" -- Terminal shortcuts for R scripts --
tnoremap ZD quarto::quarto_render(
  \output_format = "pdf")<CR>
```

```
tnoremap ZO source("<C-W>"#")
tnoremap ZQ q('no')<C-\><C-n>:q!<CR>
tnoremap ZR render("<C-W>"#")
tnoremap ZS style_dir()<CR>
tnoremap ZX exit<CR>
tnoremap ZZ q('no')<C-\><C-n>:q!<CR>

" -- Object inspection under cursor --
nnoremap <localleader>d
  \ :let @c=expand("<cword>") \|
  \ :let @d="dim(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>
nnoremap <localleader>h
  \ :let @c=expand("<cword>") \|
  \ :let @d="head(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>
nnoremap <localleader>s
  \ :let @c=expand("<cword>") \|
  \ :let @d="str(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>
nnoremap <localleader>p
  \ :let @c=expand("<cword>") \|
  \ :let @d="print(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>
nnoremap <localleader>n
  \ :let @c=expand("<cword>") \|
  \ :let @d="names(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>

" -- Auto-open R terminal --
autocmd BufEnter *
  \ if &ft ==# 'r' && !exists('b:entered')
  \ | execute(
  \   'let b:entered = 1 | :ter ++rows=5 R')
  \ | endif
```

8 UltiSnips Configuration

The UltiSnips plugin provides snippet expansion triggered by `Ctrl-Tab`. The critical configuration detail is avoiding conflicts between UltiSnips and popup menu navigation, since both want to use

the Tab key.

The solution is to assign `Ctrl-Tab` as the expand trigger and reserve plain `Tab` and `Shift-Tab` for jump-forward and jump-backward between snippet tabstops. Separately, the popup menu mappings in .vimrc use `Tab` and `Shift-Tab` only when a popup is visible.

```
let g:UltiSnipsExpandTrigger="<C-tab>"
let g:UltiSnipsJumpForwardTrigger="<tab>"
let g:UltiSnipsJumpBackwardTrigger="<S-tab>"

inoremap <expr> <tab>
  \ pumvisible() ? "\<C-n>" : "\<tab>"
inoremap <expr> <S-tab>
  \ pumvisible() ? "\<C-p>" : "\<S-tab>"
```

With this configuration, the workflow is:

1. Type a snippet trigger word (e.g., `rheader`).
2. Press `Ctrl-Tab` to expand the snippet.
3. Press `Tab` to jump to the next tabstop.
4. When a popup menu appears during typing, `Tab` navigates forward through completions instead.

9 Practical Application

This section walks through a concrete example of using the configured environment. The goal is to perform a logistic regression on the Palmer Penguins dataset, predicting gender, using only Vim and the R terminal.

Step 1: Create a working directory and open an empty R Markdown file.

```
cd ~/prj/penguins
vim -u ~/.vimrc p.Rmd
```

Step 2: Enter insert mode and type the first R command to load the data.

```
i
library(palmerpenguins)
```

Step 3: Exit insert mode with `Ctrl-C`, then yank the line into the unnamed register.

```
<C-c>
yy
```

Step 4: The ftplugin for R Markdown automatically opens an R terminal in a split pane. If it has not opened yet, use `:ter R` to start one manually.

Step 5: In the terminal pane, paste the yanked line from the register:

```
<C-w>""
```

This sends `library(palmerpenguins)` to the R REPL and executes it.

Step 6: Add plugins to .vimrc for UltiSnips support. Install the vim-plug manager if not already present:

```
curl -fLo ~/.vim/autoload/plug.vim \
  --create-dirs \
  https://raw.githubusercontent.com/\
junegunn/vim-plug/master/plug.vim
```

Step 7: Use UltiSnips to scaffold the R Markdown header. Type `rheader` on the first line and press `Ctrl-Tab`. The snippet expands into a YAML header template with tabstops for the project name, title, author, and bibliography path. Press `Tab` to navigate between tabstops. Do not leave insert mode while navigating or the snippet session will end.

10 R Markdown Template Structure

For reference, a minimal R Markdown YAML header for PDF output with LaTeX customisation looks like this:

```
---
title: "Penguins data analysis"
author: "R.G. Thomas"
date: today
output:
  pdf_document:
    keep_tex: true
header-includes:
  - \usepackage{lipsum, fancyhdr,
      titling, currfile}
  - \usepackage[export]{adjustbox}
  - \pagestyle{fancy}
---
```

A useful tip for file completion in Vim when editing R Markdown or Quarto files: temporarily set the filetype to `tex` with `:set filetype=tex`, then type `\includegraphics{` or `\input{` followed by `Ctrl-X Ctrl-O` for omni-completion of file paths.

10.1 Things to Watch Out For

1. **Tab key conflicts.** UltiSnips and popup menu completion both use Tab. If snippets stop expanding, check that `g:UltiSnipsExpandTrigger` is set to `<C-tab>` rather than `<tab>`.

2. **ftplugin not loading.** If the R terminal does not open automatically, verify that `filetype plugin indent on` appears in your .vimrc before any `Plug` calls, and that the ftplugin files are in `~/.vim/ftplugin/` (not `~/.vim/plugin/`).
3. **ALE styler dependency.** The ALE fixer for R uses the `styler` package. If ALE reports errors on save, install styler in R first: `install.packages("styler")`.
4. **Colour scheme not found.** If Vim reports `E185: Cannot find color scheme`, run `:PlugInstall` to ensure the colour scheme plugin has been downloaded.
5. **Terminal register paste.** The `<C-w>""` paste in terminal mode pastes from the unnamed register. If you yanked into a named register, use `<C-w>"a` (for register `a`).

Figure 3: A calm library reading room with natural light filtering through tall windows.

Stepping back to reflect on what was learnt.

10.2 Lessons Learnt

10.2.1 Conceptual Understanding

- A Vim-based R and LaTeX workflow replaces application switching with buffer switching, which is measurably faster once muscle memory develops.

- Filetype plugins are the correct mechanism for language-specific settings in Vim; global .vimrc settings should remain language-agnostic.
- The separation between snippet expansion (Ctrl-Tab) and popup navigation (Tab) is essential for avoiding conflicts in a multi-plugin environment.
- UltiSnips tabstops provide a structured way to scaffold repetitive code patterns, reducing boilerplate errors in YAML headers.

10.2.2 Technical Skills

- Configuring vim-plug with inline `let` statements keeps plugin settings co-located with their declarations, improving maintainability.
- The `term_sendkeys()` function in Vim 8+ enables programmatic communication with terminal buffers, which is the foundation for REPL integration.
- ALE fixers configured per filetype (`r`, `rmd`, `quarto`) apply consistent code style without manual intervention.
- The `autocmd BufEnter` pattern with a guard variable (`b:entered`) prevents multiple R terminals from spawning when switching buffers.

10.2.3 Gotchas and Pitfalls

- Leaving insert mode during UltiSnips tabstop navigation silently terminates the snippet session, leaving partially expanded text.
- The `autochdir` setting can interfere with relative file paths in R Markdown documents if the working directory changes unexpectedly.
- ALE's `ale_fix_on_save` may conflict with files that are intentionally unformatted (e.g., raw data files opened in Vim).
- The `set iskeyword-=.` setting breaks word boundaries at dots, which is helpful for R (where `data.frame` is two words) but can confuse navigation in other filetypes.

10.3 Limitations

- This configuration is specific to Vim 8+ with terminal support; Neovim requires different terminal API calls (`jobsend` instead of `term_sendkeys`).
- The ftplugin auto-opens R in a 5-row terminal split, which may be too small on low-resolution displays.
- ALE's `styler` fixer requires the `styler` R package to be installed in the active R library; if missing, ALE silently fails to fix on save.
- UltiSnips requires Vim compiled with Python 3 support; the default macOS system Vim may lack this.
- The configuration does not include debugging support; stepping through R code requires a separate tool or plugin.
- Vimtex's forward and inverse search with a PDF viewer (e.g., Zathura or Skim) requires additional configuration not covered here.

10.4 Opportunities for Improvement

1. Add Neovim-compatible terminal integration using `vim.fn.jobsend()` for cross-editor portability.
2. Configure vimtex forward search with Skim or Zathura for real-time PDF preview during LaTeX editing.
3. Create project-specific `.vimrc` files that override the terminal split size based on the display resolution.
4. Develop additional UltiSnips templates for common R analysis patterns (ggplot scaffolds, model fitting boilerplate, knitr chunk options).
5. Integrate vim-slime as an alternative to the built-in terminal for sending code to tmux sessions.
6. Add a Quarto-specific ftplugin file for `.qmd` files with render commands that target both HTML and PDF output.

11 Wrapping Up

The configuration described in this post transforms Vim into a unified environment for R programming and LaTeX document preparation. The core insight is that filetype plugins and a carefully managed Tab key mapping are the two pillars of the entire setup.

What I learnt most from this process is that configuration is iterative. The .vimrc I use today looks nothing like the one I started with. Each problem I encountered – a Tab conflict, a missing colour scheme, a terminal that would not open – taught me something about how Vim dispatches events and how plugins interact.

For anyone attempting a similar setup, I would suggest starting with just vimtex and one ftplugin file. Get the R terminal auto-opening reliably before adding UltiSnips or ALE. Layering complexity gradually makes debugging much simpler.

Main takeaways:

- Use `Ctrl-Tab` for snippet expansion to avoid Tab key conflicts with popup menu navigation.
- Place filetype-specific settings in `~/.vim/ftplugin/` rather than cluttering the global .vimrc.
- ALE with `styler` provides automatic R code formatting on save with no manual intervention.
- The `term_sendkeys()` function enables sending arbitrary R commands from normal mode to a running R REPL.

12 See Also

Related posts:

- "Setting up a minimal Neovim configuration for data science" (companion post on Neovim plugin installation)
- "Configure the Command Line for Data Science Development" (terminal and shell configuration)

Key resources:

- vimtex documentation
- UltiSnips repository
- ALE (Asynchronous Lint Engine)
- vim-plug plugin manager
- Vimtex and Zathura setup guide
- UltiSnips Python interpolation (Vimcasts)
- Latexmk documentation

13 Reproducibility

This configuration was developed and tested on macOS with Vim 9.0 and R 4.4. The following files constitute the complete configuration:

File	Purpose
`~/.vimrc`	Global plugin and editor settings
`~/.vim/ftplugin/rmd.vim`	R Markdown keybindings
`~/.vim/ftplugin/r.vim`	R script keybindings

To reproduce the environment:

```
# Install vim-plug
curl -fLo ~/.vim/autoload/plug.vim \
  --create-dirs \
  https://raw.githubusercontent.com/\
junegunn/vim-plug/master/plug.vim

# Copy configuration files
cp vimrc ~/.vimrc
cp ftplugin/rmd.vim ~/.vim/ftplugin/rmd.vim
cp ftplugin/r.vim ~/.vim/ftplugin/r.vim

# Install plugins
vim +PlugInstall +qall

# Install R dependencies
Rscript -e "install.packages('styler')"
```

14 Let's Connect

- **GitHub:** rgt47
- **Twitter/X:** @rgt47
- **LinkedIn:** Ronald Glenn Thomas
- **Email:** rgtlab.org/contact

I would enjoy hearing from you if:

- You spot an error or a better approach to any of the code in this post.
- You have suggestions for topics you would like to see covered.
- You want to discuss R programming, data science, or reproducible research.
- You have questions about anything in this tutorial.
- You just want to say hello and connect.